Regular Progress Report: Cloud-Based Movie Streaming Platform Using FastAPI and Next.js

Mustakim Ahmed Hasan (2421349042), MD. Nafiz Al Mamun (2232050642),
Tanim Ahmed (2312597042), Ahamed Osman Asif (2031080642)
CSE299.4; Group: 01
Department of Computer Science and Engineering
North South University

***Abstract*—This report summarizes the development and deployment progress of the cloud-based movie streaming platform. During this reporting period, major milestones included completion of authentication modules, movie APIs, HLS playback integration, cloud storage migration, and frontend deployment. Significant effort was dedicated to resolving build failures, CORS issues, database connectivity problems, and cloud runtime configuration. The system is currently functionally deployable with ongoing reliability tuning.**

I. Introduction

This progress report documents the implementation status of the full-stack streaming platform. The project aims to simulate a real-world cloud-based streaming system using modern frontend and backend technologies.

The reporting period focused on system stabilization, production build consistency, and resolving cloud deployment challenges.

II. Problem Statement

Although core features were implemented locally, several technical challenges emerged during deployment:

- TypeScript build failures during cloud frontend builds.
- Mismatch between frontend and backend API field naming.
- Large repository push failures due to tracked build artifacts.
- Backend runtime crashes in cloud containers.
- CORS inconsistencies across frontend-backend domains.
- Cloud PostgreSQL SSL and async connectivity issues.

The goal during this period was to resolve these issues and transition from local prototype to stable cloud deployment.

III. Methods

The work during this period was divided into iterative stabilization phases.

A. Iteration 1: Core Feature Completion

Completed:

- FastAPI modular routes: `/auth`, `/movies`, `/history`.
- SQLAlchemy async model integration.
- Next.js pages for home, search, details, authentication, and history.
- HLS.js integration for adaptive playback.

End-to-end functionality worked locally.

B. Iteration 2: Frontend Build Stabilization

Issues Identified:

- TypeScript strict-mode failures during production builds.
- SWR typing mismatches.
- camelCase and snake_case field inconsistencies.

Solutions Applied:

- Explicit frontend response typing.
- Standardization to backend schema fields (`hls_master_url`, `duration_minutes`, `maturity_rating`).
- Conditional rendering guards for undefined data.

Frontend production builds became stable.

C. Iteration 3: Media and Backend Deployment

Issues Identified:

- GitHub push rejection due to tracked build artifacts.
- Backend import and entrypoint misconfiguration.
- Static media folder absence in cloud container.

Solutions Applied:

- Removed generated artifacts and updated ignore policy.
- Corrected backend startup configuration.
- Migrated video content to Supabase Storage.
- Implemented conditional static folder mounting.

Deployment model transitioned to cloud-hosted media architecture.

D. Iteration 4: CORS and Database Hardening

Issues Identified:

- Intermittent 500 and 422 backend responses.
- CORS failures despite valid later retries.
- Cloud PostgreSQL SSL/pooler connection constraints.

Solutions Applied:

- Resolved route declaration conflicts.
- Normalized CORS origin parsing.
- Updated asyncpg-compatible DB connection settings.
- Moved configuration to environment variables.

System reached stable operational state.

IV. Expected Outcomes

At the end of this reporting period, the system:

- Successfully supports authentication and session flow.
- Provides movie listing, search, details, and recommendations.
- Streams video via HLS from cloud storage.
- Stores and retrieves watch history from PostgreSQL.
- Deploys frontend and backend independently in cloud.

Remaining improvements focus on reliability tuning and request retry optimization.

V. Resources

Technologies and tools used during this period:

- Next.js with TypeScript
- FastAPI
- SQLAlchemy (async)
- PostgreSQL (cloud-hosted)
- Supabase Storage for HLS media
- HLS.js for playback
- GitHub for version control
- Cloud hosting platform for deployment

Reference documentation:

- FastAPI: https://fastapi.tiangolo.com
- Next.js: https://nextjs.org/docs
- SQLAlchemy: https://docs.sqlalchemy.org
- Supabase: https://supabase.com/docs